LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
ENKAPSULASI



Identitas Mahasiswa :

Nama	: Reivan Haikal Sulaeman
NIM	: 2530521091
Kelas	: 3 TI C

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS MUHAMMADIYAH SUKABUMI
2026

1. Tujuan dan Ringkasan Konsep

Tujuan Praktikum

Praktikum Pertemuan 5 bertujuan agar mahasiswa mampu menjelaskan konsep enkapsulasi, membedakan ruang lingkup akses public, protected, dan private, membuat getter, setter, serta validasi untuk menjaga aturan bisnis objek, dan menggunakan constructor agar objek valid sejak dibuat. Seluruh konsep tersebut diterapkan pada refactoring aplikasi CRUD Data Mahasiswa sehingga property tidak lagi diakses langsung dari index.php.

Ringkasan Konsep Enkapsulasi

Enkapsulasi adalah pengemasan data beserta perilaku yang mengelola data tersebut ke dalam satu class, disertai pengendalian cara pihak luar mengakses atau mengubahnya. Analoginya seperti brankas: isi (state objek) disimpan di dalam, sedangkan orang luar hanya bisa memakai "kunci" yang disediakan, yaitu method publik. Dengan begitu aturan seperti semester harus 1 sampai 14 tidak bisa dilewati begitu saja.

Di PHP, tingkat akses diatur oleh tiga modifier. Public dapat diakses dari mana saja. Protected dapat diakses dari class itu sendiri dan class turunannya. Private hanya dapat diakses dari dalam class yang mendeklarasikannya. Property yang bersifat private dibaca melalui getter dan diubah melalui setter yang memeriksa aturan terlebih dahulu. Penting dicatat bahwa enkapsulasi bukan sekadar mengganti public menjadi private; method berbasis perilaku seperti tambahStok() atau setor() sering lebih tepat daripada setter bebas, dan tidak semua property wajib memiliki getter maupun setter.

2. Cuplikan kode sebelum dan sesudah Refactoring

Sebelum Refactoring (Pertemuan 4)

Seluruh property pada class Mahasiswa bersifat public sehingga dapat dibaca dan diubah dari mana saja tanpa pemeriksaan apa pun.

```
class Mahasiswa {
   public string $nim;
   public string $nama;
   public int $semester;
   // ... property lain juga public
}

$m->semester = -7;                // lolos tanpa validasi
$this->data[$m->nim] = $m;        // repository membaca property langsung
<td><?= aman($m->nama) ?></td>    // index.php membaca property langsung
```

Sesudah Refactoring (Pertemuan 5)

Property diubah menjadi private. Nilai awal ditetapkan lewat constructor yang memakai kembali setter, sehingga aturan validasi hanya ditulis satu kali. NIM tidak memiliki setter karena diperlakukan sebagai identitas tetap.

```
class Mahasiswa {
   private string $nim;
   private string $nama;
   private int $semester;

   public function __construct(string $nim, string $nama, int $semester, /* ... */) {
      $nim = trim($nim);
      if ($nim === "") throw new InvalidArgumentException("NIM wajib diisi");
      $this->nim = $nim;              // tanpa setNim()
      $this->setNama($nama);
      $this->setSemester($semester);
   }

   public function getNim(): string      { return $this->nim; }
   public function getSemester(): int    { return $this->semester; }

   public function setSemester(int $semester): void {
      if ($semester < 1 || $semester > 14)
         throw new InvalidArgumentException("Semester harus berada pada rentang 1-14");
      $this->semester = $semester;
   }
}

$this->data[$m->getNim()] = $m;          // DataMahasiswa.php
<td><?= aman($m->getNama()) ?></td>      // index.php
```

Penjelasan Perubahan

Setelah property menjadi private, kode di luar class tidak dapat lagi memakai $m->nim, $m->nama, atau $m->semester karena PHP akan menghasilkan error akses. Karena itu DataMahasiswa.php diubah agar mengambil key lewat getNim(), dan index.php diubah agar menampilkan data serta mengisi form edit lewat getter (misalnya $edit?->getNama()). Perubahan data hanya bisa lewat setter atau constructor, sehingga nilai seperti semester -7 langsung ditolak oleh exception. Pada fitur Edit, NIM ditampilkan read-only dan repository memperbarui object pada key NIM yang sama.

3. Screenshot Aplikasi CRUD dan Tabel Hasil Pengujian

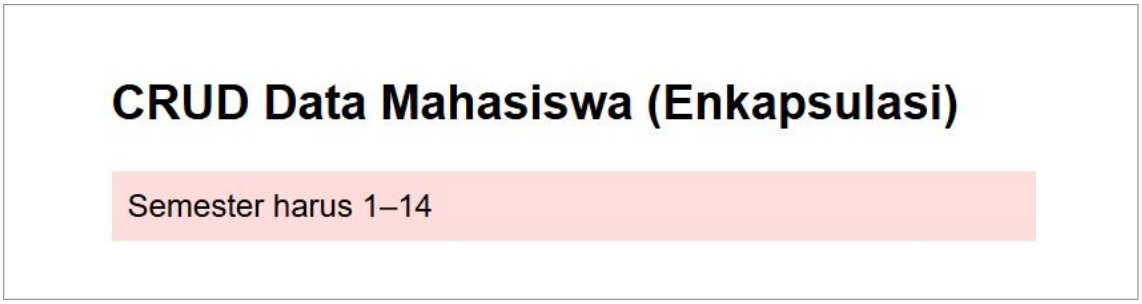
Pengujian dilakukan pada aplikasi CRUD Mahasiswa berbasis PHP Session (tanpa database) yang dijalankan melalui XAMPP di http://localhost/pbo05-enkapsulasi/index.php.



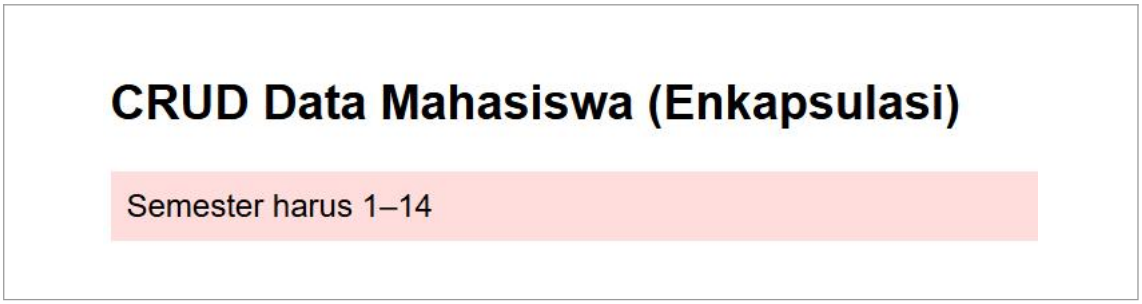
Gambar 1 Mahasiswa valid, semester 4



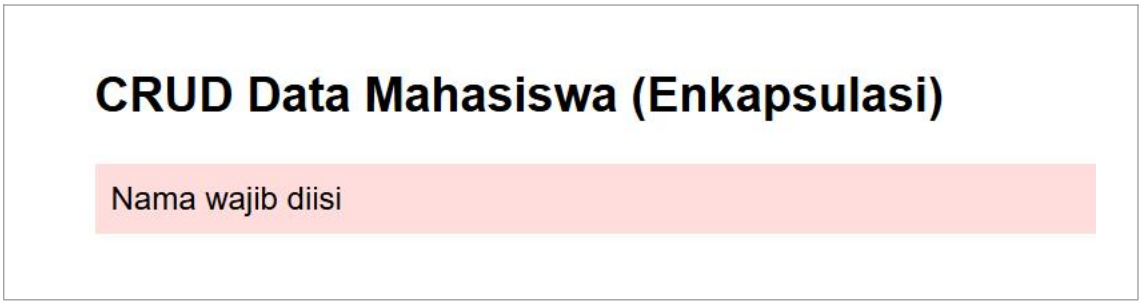
Gambar 2 NIM kosong



Gambar 3 Semester -1



Gambar 4 Semester 15



Gambar 5 Nama kosong/spasi



Gambar 6 Edit nama valid



Gambar 7 NIM duplikat



Gambar 8 Hapus mahasiswa



Gambar 9 Data tetap ada setelah refresh

Tabel Hasil Pengujian

No	Kasus Uji	Input/Aksi	Hasil yang Diharapkan
1	Mahasiswa valid	Semester 4, semua kolom terisi benar	Object dibuat dan tersimpan di Session, tampil di tabel.
2	NIM kosong	Kolom NIM dikosongkan lalu simpan	Ditolak, pesan "Error: NIM wajib diisi".
3	Semester -1	Isi semester dengan -1	Ditolak oleh setter, pesan "Error: Semester harus berada pada rentang 1-14".
4	Semester 15	Isi semester dengan 15	Ditolak oleh setter, pesan "Error: Semester harus berada pada rentang 1-14".
5	Nama kosong/spasi	Kolom nama hanya berisi spasi	Ditolak oleh setter, pesan "Error: Nama tidak boleh kosong".
6	Akses langsung $m->semester	Dicoba pada skrip uji terpisah	Fatal error: Cannot access private property Mahasiswa::$semester.
7	Edit nama valid	Ubah nama pada form edit	Nama berubah melalui setNama(); NIM tetap read-only.

No	Kasus Uji	Input/Aksi	Hasil yang Diharapkan
8	NIM duplikat pada tambah	Tambah dengan NIM yang sudah ada	Ditolak oleh repository, pesan "Error: NIM 2530521091 sudah terdaftar".
9	Hapus mahasiswa	Klik Hapus pada satu baris	Hanya record terkait yang dihapus, data lain tetap ada.
10	Refresh browser	Muat ulang halaman setelah tambah data	Data tetap tersedia selama Session berlaku.

4. Analisis Kasus

Analisis Error Akses Private

Pada skrip uji, baris echo $m->semester; dipanggil dari luar class Mahasiswa. Karena semester dideklarasikan private, PHP menghentikan program dengan pesan Fatal error: Uncaught Error: Cannot access private property Mahasiswa::$semester. Ini menunjukkan bahwa penjagaan dilakukan oleh bahasa itu sendiri, bukan oleh kesepakatan programmer. Untuk membaca nilainya, kode luar harus memanggil getSemester(), dan untuk mengubahnya harus memanggil setSemester() sehingga selalu melewati pemeriksaan.

Analisis Validasi Input

Ketika form diisi semester 15, index.php memanggil constructor Mahasiswa, lalu constructor meneruskan nilai tersebut ke setSemester(). Nilai di luar rentang 1-14 membuat setter melempar InvalidArgumentException. Exception ditangkap oleh blok catch pada index.php dan pesannya ditampilkan sebagai Error di atas form. Karena exception dilempar sebelum object selesai dibuat, perintah $repo->tambah() tidak dijalankan, sehingga data yang tidak valid tidak pernah masuk ke Session. Pada proses edit, perubahan dilakukan pada salinan object (clone) dan baru disimpan jika semua setter lolos, supaya data lama tidak setengah berubah ketika salah satu isian ditolak.

5. Jawaban Tugas Praktikum

Tugas 1 — Analisis access modifier (individu)

Buat class Produk dengan properti public nama, protected kategori, dan private harga. Uji akses ketiganya dari luar class. Catat hasil dan jelaskan mengapa akses tertentu ditolak. Untuk menguji protected dari class turunan, buat satu subclass ProdukDigital.

Kode program :

```
<?php
// Tugas 1 - Analisis access modifier
class Produk {
   public string $nama;
   protected string $kategori;
```

```
    private int $harga;

    public function __construct(string $nama, string $kategori, int $harga) {
        $this->nama = $nama;
        $this->kategori = $kategori;
        $this->harga = $harga;
    }
    public function getHarga(): int { return $this->harga; }
}

class ProdukDigital extends Produk {
    public function info(): string {
        // protected: BOLEH dari class turunan
        $r = "Kategori (protected, dari subclass): {$this->kategori}<br>";
        // private milik parent: TIDAK dapat diakses langsung dari subclass
        // echo $this->harga;  // -> Warning: Undefined property (private milik Produk)
        $r .= "Harga lewat getter: " . $this->getHarga() . "<br>";
        return $r;
    }
}

$p = new ProdukDigital("E-Book PBO", "Buku Digital", 50000);

echo "<h3>Uji akses dari luar class</h3>";
echo "1. public nama: " . $p->nama . " -> BERHASIL<br>";

try { echo $p->kategori; }
catch (Error $e) { echo "2. protected kategori -> DITOLAK: " . $e->getMessage() .
"<br>"; }

try { echo $p->harga; }
catch (Error $e) { echo "3. private harga -> DITOLAK: " . $e->getMessage() .
"<br>"; }

echo "<h3>Uji dari class turunan (ProdukDigital)</h3>";
echo $p->info();
```

Hasil pengujian :



Gambar 10 Hasil pengujian access modifier

Property	Modifier	Akses dari luar class	Hasil
$nama	public	Langsung ($p->nama)	Berhasil dibaca dan diubah.
$kategori	protected	Langsung ($p->kategori)	Fatal error: Cannot access protected property ProdukDigital::$kategori.
$kategori	protected	Lewat infoKategori() pada subclass	Berhasil
$harga	private	Langsung ($p->harga)	Fatal error: Cannot access private property ProdukDigital::$harga.
$harga	private	Lewat getHarga() pada class Produk	Berhasil: Rp 50.000

Penjelasan :

Property $nama berstatus public sehingga siapa pun dapat membaca dan mengubahnya, termasuk menimpa nilainya tanpa pemeriksaan. Property $kategori berstatus protected: dari luar class aksesnya ditolak, tetapi method di dalam ProdukDigital boleh memakainya karena subclass berada dalam rantai pewarisan. Property $harga berstatus private sehingga hanya kode di dalam class Produk yang boleh menyentuhnya; subclass pun tidak bisa membacanya secara langsung. Itulah sebabnya method cobaHarga() pada subclass akan gagal, dan harga hanya bisa diperoleh lewat getHarga() yang dibuat di class induk. Penolakan ini terjadi karena PHP menegakkan aturan visibilitas saat program berjalan, sehingga detail internal objek terlindungi.

Tugas 2 — Getter, setter, dan validasi (individu)

Buat class Buku dengan properti private kodeBuku, judul, stok. Kode buku hanya dapat dibaca; judul wajib diisi; stok tidak boleh negatif. Buat constructor, getter yang diperlukan,

setJudul(), tambahStok(), dan kurangiStok() dengan pemeriksaan stok mencukupi. Uji setidaknya dua input valid dan tiga input tidak valid.

Kode program :

```
<?php
declare(strict_types=1);

class Buku {
   private string $kodeBuku;
   private string $judul;
   private int $stok;

   public function __construct(string $kodeBuku, string $judul, int $stok) {
      $kodeBuku = trim($kodeBuku);
      if ($kodeBuku === "") throw new InvalidArgumentException("Kode buku
wajib diisi");
      $this->kodeBuku = $kodeBuku;      // read-only: tidak ada setKodeBuku()
      $this->setJudul($judul);
      if ($stok < 0) throw new InvalidArgumentException("Stok awal tidak boleh
negatif");
      $this->stok = $stok;
   }
   public function getKodeBuku(): string { return $this->kodeBuku; }
   public function getJudul(): string { return $this->judul; }
   public function getStok(): int { return $this->stok; }

   public function setJudul(string $judul): void {
      $judul = trim($judul);
      if ($judul === "") throw new InvalidArgumentException("Judul wajib diisi");
      $this->judul = $judul;
   }
   public function tambahStok(int $jumlah): void {
      if ($jumlah <= 0) throw new InvalidArgumentException("Jumlah tambah stok
harus positif");
      $this->stok += $jumlah;
   }
   public function kurangiStok(int $jumlah): void {
      if ($jumlah <= 0) throw new InvalidArgumentException("Jumlah pengurangan
harus positif");
      if ($jumlah > $this->stok) throw new InvalidArgumentException("Stok tidak
mencukupi (sisa {$this->stok})");
      $this->stok -= $jumlah;
   }
}

function uji(string $label, callable $aksi): void {
   try { $aksi(); echo "[VALID]   $label -> berhasil<br>"; }
```

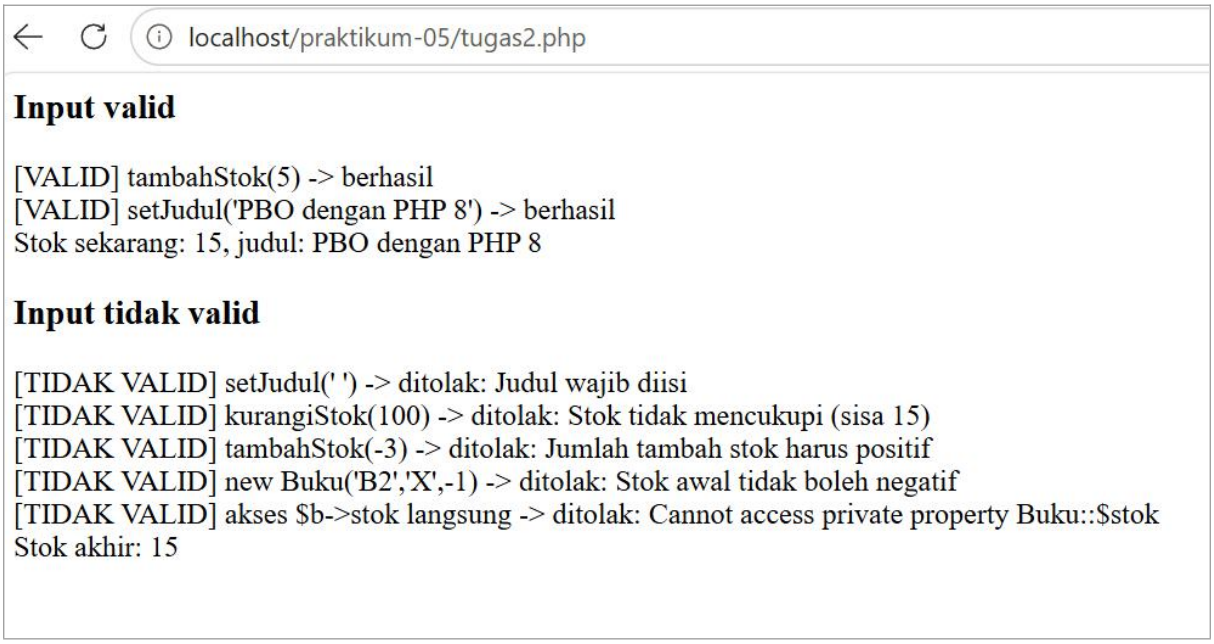
```
   catch (Throwable $e) { echo "[TIDAK VALID] $label -> ditolak: " . $e-
>getMessage() . "<br>"; }
}

$b = new Buku("BK001", "Pemrograman PHP", 10);

echo "<h3>Input valid</h3>";
uji("tambahStok(5)", fn() => $b->tambahStok(5));
uji("setJudul('PBO dengan PHP 8')", fn() => $b->setJudul("PBO dengan PHP 8"));
echo "Stok sekarang: {$b->getStok()}, judul: {$b->getJudul()}<br>";

echo "<h3>Input tidak valid</h3>";
uji("setJudul('   ')", fn() => $b->setJudul("    "));
uji("kurangiStok(100)", fn() => $b->kurangiStok(100));
uji("tambahStok(-3)", fn() => $b->tambahStok(-3));
uji("new Buku('B2','X',-1)", fn() => new Buku("B2", "X", -1));
uji("akses \$b->stok langsung", fn() => $b->stok = -99);
echo "Stok akhir: {$b->getStok()}<br>";
```

Hasil pengujian :



Gambar 11 Hasil pengujian class Buku

Penjelasan :

Ketiga property class Buku bersifat private sehingga tidak dapat diubah dari luar. Kode buku sengaja hanya memiliki getter karena berfungsi sebagai identitas yang tidak berubah. Judul

diubah lewat setJudul() yang menolak teks kosong atau hanya spasi. Stok tidak diberi setStok() bebas; perubahannya hanya lewat tambahStok() dan kurangiStok(), dan kurangiStok() memeriksa bahwa jumlah tidak melebihi stok tersedia. Pada pengujian, dua operasi valid menghasilkan stok 15 (8 + 12 - 5), sedangkan empat input tidak valid ditolak dengan InvalidArgumentException dan stok tetap 15. Hal ini membuktikan state objek tidak berubah ketika input melanggar aturan.

Tugas 3 — Refactoring CRUD Mahasiswa (kelompok kecil)

Ubah aplikasi Pertemuan 4 sehingga properti nim, nama, fakultas, prodi, asalSekolah, semester, hobi, dan agama tidak dapat diakses langsung dari index.php. Buat getter untuk tampilan dan setter hanya bagi atribut yang memang boleh diubah. Pertahankan operasi tambah, tampil, edit, hapus. Validasi semester 1–14, NIM wajib dan unik, nama tidak kosong, serta pilihan agama harus termasuk daftar yang diizinkan.
Ketentuan tambahan: NIM ditetapkan tetap setelah object dibuat. Pada fitur Edit, tampilkan NIM sebagai read-only atau non-editable; jangan menyediakan setNim() tanpa rancangan konsistensi key repository. Data tetap menggunakan PHP Session, bukan database.

Kode program :

Mahasiswa.php (ringkas)

```
<?php
declare(strict_types=1);

class Mahasiswa {
   public const AGAMA = ['Islam', 'Kristen', 'Katolik', 'Hindu', 'Buddha',
'Konghucu'];

   private string $nim;
   private string $nama;
   private string $fakultas;
   private string $prodi;
   private string $asalSekolah;
   private int $semester;
   private string $hobi;
   private string $agama;

   public function __construct(
      string $nim, string $nama, string $fakultas, string $prodi,
      string $asalSekolah, int $semester, string $hobi, string $agama
   ) {
      $nim = trim($nim);
      if ($nim === "") throw new InvalidArgumentException("NIM wajib diisi");
      $this->nim = $nim; // identitas tetap: tidak ada setNim()
      $this->setNama($nama);
      $this->setFakultas($fakultas);
      $this->setProdi($prodi);
      $this->setAsalSekolah($asalSekolah);
```

```
        $this->setSemester($semester);
        $this->setHobi($hobi);
        $this->setAgama($agama);
    }

    public function getNim(): string { return $this->nim; }
    public function getNama(): string { return $this->nama; }
    public function getFakultas(): string { return $this->fakultas; }
    public function getProdi(): string { return $this->prodi; }
    public function getAsalSekolah(): string { return $this->asalSekolah; }
    public function getSemester(): int { return $this->semester; }
    public function getHobi(): string { return $this->hobi; }
    public function getAgama(): string { return $this->agama; }

    private static function wajib(string $nilai, string $label): string {
        $nilai = trim($nilai);
        if ($nilai === "") throw new InvalidArgumentException("$label wajib diisi");
        return $nilai;
    }
    public function setNama(string $v): void { $this->nama = self::wajib($v,
"Nama"); }
    public function setFakultas(string $v): void { $this->fakultas = self::wajib($v,
"Fakultas"); }
    public function setProdi(string $v): void { $this->prodi = self::wajib($v,
"Prodi"); }
    public function setAsalSekolah(string $v): void { $this->asalSekolah =
self::wajib($v, "Asal sekolah"); }
    public function setHobi(string $v): void { $this->hobi = trim($v); } // hobi boleh
kosong
    public function setSemester(int $v): void {
        if ($v < 1 || $v > 14) throw new InvalidArgumentException("Semester harus 1–
14");
        $this->semester = $v;
    }
    public function setAgama(string $v): void {
        $v = trim($v);
        if (!in_array($v, self::AGAMA, true)) throw new
InvalidArgumentException("Agama tidak termasuk daftar yang diizinkan");
        $this->agama = $v;
    }
}
```

DataMahasiswa.php

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/Mahasiswa.php';

class DataMahasiswa {
   private array $data; // [nim => Mahasiswa], disimpan di $_SESSION

   public function __construct() {
      $_SESSION['mahasiswa'] ??= [];
      $this->data = &$_SESSION['mahasiswa'];
   }
   public function tambah(Mahasiswa $m): void {
      if (isset($this->data[$m->getNim()])) {
         throw new InvalidArgumentException("NIM {$m->getNim()} sudah
terdaftar");
      }
      $this->data[$m->getNim()] = $m;
   }
   public function semua(): array { return $this->data; }
   public function cari(string $nim): ?Mahasiswa { return $this->data[$nim] ?? null; }

   /** NIM tidak diubah; objek baru dibangun (divalidasi) lalu ditimpa pada key yang
sama. */
   public function ubah(string $nim, string $nama, string $fakultas, string $prodi,
                        string $asalSekolah, int $semester, string $hobi, string $agama): void
{
      if (!isset($this->data[$nim])) throw new InvalidArgumentException("Data tidak
ditemukan");
      $baru = new Mahasiswa($nim, $nama, $fakultas, $prodi, $asalSekolah,
$semester, $hobi, $agama);
      $this->data[$nim] = $baru;
   }
   public function hapus(string $nim): void { unset($this->data[$nim]); }
}
```

index.php (bagian penanganan input dan tampilan)

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/DataMahasiswa.php'; // class dimuat sebelum session_start
session_start();

function aman(mixed $s): string { return htmlspecialchars((string)$s,
```

```
ENT_QUOTES, 'UTF-8'); }
function in(string $k): string { return trim((string)($_POST[$k] ?? '')); }

$repo = new Data Mahasiswa();
$error = $_SESSION['flash_error'] ?? null;
$info  = $_SESSION['flash_info'] ?? null;
unset($_SESSION['flash_error'], $_SESSION['flash_info']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $aksi = in('aksi');
    try {
        if ($aksi === 'hapus') {
            $repo->hapus(in('nim'));
            $_SESSION['flash_info'] = "Data dihapus";
        } else {
            $sem = filter_var(in('semester'), FILTER_VALIDATE_INT);
            if ($sem === false) throw new InvalidArgumentException("Semester harus
berupa angka bulat");
            $args = [in('nim'), in('nama'), in('fakultas'), in('prodi'),
                    in('asalSekolah'), $sem, in('hobi'), in('agama')];
            if ($aksi === 'tambah') {
                $repo->tambah(new Mahasiswa(...$args));
                $_SESSION['flash_info'] = "Data berhasil ditambahkan";
            } elseif ($aksi === 'ubah') {
                $repo->ubah(...$args);
                $_SESSION['flash_info'] = "Data berhasil diubah";
            }
        }
    } catch (InvalidArgumentException $e) {
        $_SESSION['flash_error'] = $e->getMessage();
        if ($aksi === 'ubah') { header('Location: index.php?edit=' . urlencode(in('nim')));
exit; }
    }
    header('Location: index.php'); exit;
}

$edit = isset($_GET['edit']) ? $repo->cari((string)$_GET['edit']) : null;
?>
<!DOCTYPE html>
<html lang="id"><head><meta charset="utf-8"><title>CRUD Mahasiswa -
Enkapsulasi</title>
<style>
body{font-family:sans-serif;max-width:960px;margin:20px auto;padding:0 12px}
table{border-collapse:collapse;width:100%}td,th{border:1px solid
#ccc;padding:6px}
label{display:block;margin:6px 0 2px}input,select{width:100%;padding:6px;box-
sizing:border-box}
```

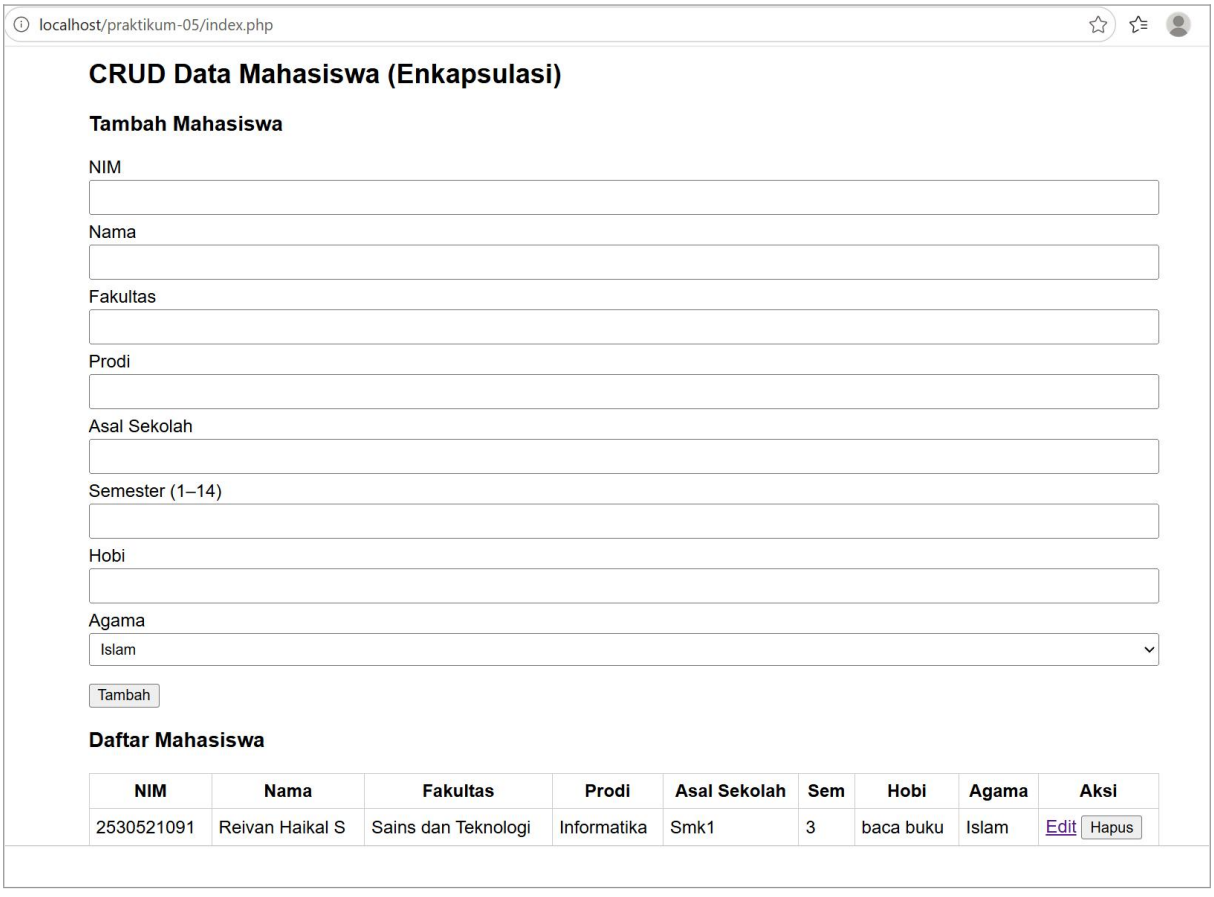
```
.err{background:#fdd;padding:8px}.ok{background:#dfd;padding:8px}
</style></head><body>
<h2>CRUD Data Mahasiswa (Enkapsulasi)</h2>
<?php if ($error): ?><p class="err"><?= aman($error) ?></p><?php endif; ?>
<?php if ($info): ?><p class="ok"><?= aman($info) ?></p><?php endif; ?>

<h3><?= $edit ? 'Edit Mahasiswa' : 'Tambah Mahasiswa' ?></h3>
<form method="post">
  <input type="hidden" name="aksi" value="<?= $edit ? 'ubah' : 'tambah' ?>">
  <label>NIM</label>
  <?php if ($edit): ?>
    <input name="nim" value="<?= aman($edit->getNim()) ?>" readonly>
  <?php else: ?>
    <input name="nim" required>
  <?php endif; ?>
  <label>Nama</label><input name="nama" value="<?= aman($edit?->getNama() ??
'') ?>">
  <label>Fakultas</label><input name="fakultas" value="<?= aman($edit?-
>getFakultas() ?? '') ?>">
  <label>Prodi</label><input name="prodi" value="<?= aman($edit?->getProdi() ??
'') ?>">
  <label>Asal Sekolah</label><input name="asalSekolah" value="<?= aman($edit?-
>getAsalSekolah() ?? '') ?>">
  <label>Semester (1–14)</label><input name="semester" type="number"
value="<?= aman($edit?->getSemester() ?? '') ?>">
  <label>Hobi</label><input name="hobi" value="<?= aman($edit?->getHobi() ??
'') ?>">
  <label>Agama</label>
  <select name="agama">
    <?php foreach (Mahasiswa::AGAMA as $a): ?>
      <option <?= ($edit?->getAgama() === $a) ? 'selected' : '' ?>><?=
aman($a) ?></option>
    <?php endforeach; ?>
  </select>
  <p><button type="submit"><?= $edit ? 'Simpan' : 'Tambah' ?></button>
  <?php if ($edit): ?><a href="index.php">Batal</a><?php endif; ?></p>
</form>

<h3>Daftar Mahasiswa</h3>
<table>
<tr><th>NIM</th><th>Nama</th><th>Fakultas</th><th>Prodi</th><th>Asal
Sekolah</th><th>Sem</th><th>Hobi</th><th>Agama</th><th>Aksi</th></tr>
<?php foreach ($repo->semua() as $m): ?>
<tr>
  <td><?= aman($m->getNim()) ?></td><td><?= aman($m->getNama()) ?></td>
  <td><?= aman($m->getFakultas()) ?></td><td><?= aman($m->getProdi()) ?></td>
  <td><?= aman($m->getAsalSekolah()) ?></td><td><?= aman($m-
```

```
>getSemester()) ?></td>
  <td><?= aman($m->getHobi()) ?></td><td><?= aman($m->getAgama()) ?></td>
  <td><a href="?edit=<?= urlencode($m->getNim()) ?>">Edit</a>
    <form method="post" style="display:inline" onsubmit="return
confirm('Hapus?')">
      <input type="hidden" name="aksi" value="hapus"><input type="hidden"
name="nim" value="<?= aman($m->getNim()) ?>">
      <button>Hapus</button></form></td>
</tr>
<?php endforeach; ?>
</table>
</body></html>
```

Hasil pengujian :



Gambar 12 Tampilan aplikasi CRUD Mahasiswa setelah refactoring

Penjelasan :

Pada refactoring ini kedelapan property class Mahasiswa dijadikan private. index.php dan DataMahasiswa.php tidak lagi menyentuh property, melainkan memakai getter untuk menampilkan data dan setter untuk mengubahnya. Setter dibuat untuk semua atribut kecuali NIM, karena NIM adalah key pada repository; mengubahnya tanpa rancangan khusus akan membuat key di Session tidak sinkron. Aturan validasi (semester 1-14, nama tidak kosong,

agama harus ada dalam daftar AGAMA_VALID, NIM wajib dan unik) ditegakkan di class dan repository, bukan di form, sehingga berlaku di mana pun class dipakai. Operasi tambah, tampil, edit, dan hapus tetap berjalan dengan data disimpan pada PHP Session.

Tugas 4 — Refleksi konsep (individu)

1. Mengapa private saja belum cukup untuk memastikan objek valid? Private hanya menutup akses langsung. Jika setter atau constructor tidak memvalidasi, nilai tidak valid tetap bisa masuk lewat jalur resmi, misalnya setSemester(-7). Objek valid membutuhkan private ditambah validasi.

2. Kapan setter tidak perlu dibuat? Ketika nilai tidak boleh berubah atau hanya boleh berubah lewat aturan khusus. Contohnya nim pada Mahasiswa (identitas tetap) dan saldo pada Rekening (hanya berubah lewat setor() dan tarik()).

3. Mengapa tambahStok() dan kurangiStok() lebih aman daripada setStok() bebas? Keduanya merepresentasikan operasi bermakna dan memeriksa aturan, yaitu jumlah harus positif dan stok harus cukup. setStok() bebas memungkinkan stok diganti sembarang nilai sehingga tidak lagi mencerminkan transaksi yang sebenarnya.

4. Apakah __get() dan __set() wajib untuk menerapkan enkapsulasi? Tidak wajib. Enkapsulasi cukup dengan access modifier, validasi, dan method yang jelas. __get() dan __set() hanyalah sarana tambahan, dan jika dipakai tanpa validasi justru dapat membuka kembali akses bebas ke properti.

5. Bagaimana perubahan public menjadi private memengaruhi DataMahasiswa.php dan index.php? Semua pemanggilan $m->nim, $m->nama, dan $m->semester harus diganti dengan $m->getNim(), $m->getNama(), dan $m->getSemester(). Repository mengambil key melalui getNim(), dan form Edit memakai $edit?->getNama() dengan operator nullsafe. Jika ada pemanggilan yang terlewat, aplikasi akan menampilkan error akses private.

.

6. Kesimpulan

Enkapsulasi menjaga konsistensi data objek dengan cara menyembunyikan state di balik properti private dan hanya membuka jalur akses yang tervalidasi, yaitu getter, setter, dan method berbasis perilaku. Pada aplikasi CRUD Mahasiswa, perubahan dari public ke private memang menuntut perubahan pada repository dan tampilan, tetapi hasilnya aturan seperti semester 1–14, NIM wajib dan unik, serta agama yang valid selalu ditegakkan dan tidak dapat dilewati dari luar class. NIM yang dibuat tetap (tanpa setter) juga menjaga key repository tetap konsisten. Dengan demikian aplikasi menjadi lebih aman, lebih mudah dirawat, dan perubahan aturan cukup dilakukan di satu tempat.

7. Link Github

reivanhaikal24-ui/PraktikumPBO5